

four.js - Unified 2D, 3D, Motion, Animation, and Physics Framework

Tagline: One scene. Every dimension. Everything moves.

Status: Revised architectural specification and implementation plan

Primary language: TypeScript

Proposed license: MIT

Target platforms: Web browsers, Web Workers, Node-compatible headless environments, and future native runtimes

1. Vision

four.js is a unified JavaScript and TypeScript framework for building interactive applications that combine:

- 2D graphics
- 2.5D scenes
- 3D graphics
- vector graphics
- raster graphics
- text and user interfaces
- animation
- motion systems
- rigid-body physics
- soft-body and particle simulation
- constraints and joints
- engineering and scientific simulation
- GPU computation

The name **four.js** represents more than “the library after Three.js.” It represents a fourth layer that brings **time, motion, animation, and physical behavior** into a unified 2D/3D scene model.

A scene is not merely a collection of objects in space. It is a system of objects changing through time.

The framework therefore treats the following as equally fundamental:

1. **Space** - where an object exists.
 2. **Appearance** - how an object is rendered.
 3. **Interaction** - how an object receives input.
 4. **Dynamics** - how an object changes, moves, collides, and responds over time.
-

2. Core Design Principle

Every visible, interactive, animated, or simulated entity is represented within one shared scene.

Scene

- Static Geometry
- Animated Objects
- Dynamic Bodies
- Constraints and Joints
- Particle Systems
- Cameras and Lights
- 2D Diagrams
- 3D Models
- User Interface

A 2D circle, 3D mesh, text label, rigid body, spring, motor, particle emitter, animation controller, and sensor visualization can participate in the same application lifecycle.

3. Four Architectural Pillars

four.js is organized around four coequal pillars.

3.1 Scene

The scene graph defines hierarchy, transforms, visibility, grouping, and ownership.

3.2 Render

The rendering system converts logical scene state into pixels through WebGPU, WebGL 2, Canvas 2D, SVG, or headless backends.

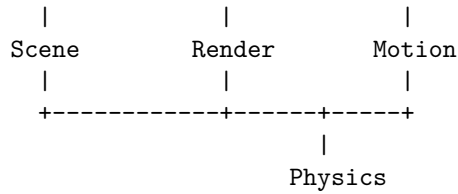
3.3 Motion

The motion system defines deterministic changes through time, including animation, velocity, acceleration, trajectories, interpolation, procedural movement, and kinematic control.

3.4 Physics

The physics system models forces, mass, collisions, constraints, impulses, joints, fields, and numerical integration.

four.js
+-----+-----+



Motion and physics are related but not identical:

- **Animation** specifies how something should move.
- **Physics** calculates how something must move under physical rules.
- **Kinematics** moves objects directly without solving forces.
- **Dynamics** derives motion from forces, mass, and constraints.

four.js must support all four approaches and allow controlled blending between them.

4. Goals

four.js shall:

1. unify 2D and 3D objects under one scene graph;
 2. make animation and motion first-class engine systems;
 3. provide a common physics API for both 2D and 3D;
 4. support deterministic fixed-step simulation;
 5. separate logical physics state from rendering backends;
 6. permit pluggable physics solvers while maintaining a stable four.js API;
 7. support rigid bodies, colliders, forces, impulses, constraints, joints, and sensors;
 8. support keyframe, procedural, skeletal, morph, path, and physics-driven animation;
 9. provide interpolation between simulation states for smooth rendering;
 10. support worker-based and GPU-accelerated simulation;
 11. enable engineering and scientific applications, not only games;
 12. support serialization, replay, debugging, and reproducible simulation.
-

5. Non-Goals

The initial release shall not attempt to provide:

- a complete industrial finite-element solver;
- a certified safety-critical physics simulator;
- a complete computational fluid-dynamics package;
- a full CAD geometric kernel;
- a full game editor;

- exact real-world simulation across all scales.

These may be supported by plugins or specialized solver integrations.

Part I - Core Scene Architecture

6. Unified Node Model

```
abstract class Node {
  readonly id: string;
  name: string;

  parent: Node | null;
  readonly children: Node[];

  readonly transform: Transform;

  visible: boolean;
  enabled: boolean;
  opacity: number;

  tags: Set<string>;
  metadata: Record<string, unknown>;

  add(...nodes: Node[]): this;
  remove(...nodes: Node[]): this;
  traverse(visitor: (node: Node) => void): void;
}
```

Every node may optionally participate in:

- rendering;
- animation;
- input;
- physics;
- layout;
- audio;
- serialization.

The base `Node` should remain lightweight. Extended behavior should be attached through typed components or specialized subclasses.

7. Transform System

Every node uses a common 3D transform representation.

```
class Transform {
  position: Vector3;
  rotation: Quaternion;
  scale: Vector3;
  pivot: Vector3;

  localMatrix: Matrix4;
  worldMatrix: Matrix4;

  matrixAutoUpdate: boolean;
  version: number;
}
```

Two-dimensional nodes normally use:

```
position.z = 0;
scale.z = 1;
```

The same transform hierarchy therefore supports:

- 2D scenes;
- 3D scenes;
- world-space UI;
- screen-space UI;
- billboards;
- planar diagrams in 3D;
- physics bodies;
- animated skeletons.

8. Space Modes

```
type SpaceMode =
  | "world"
  | "screen"
  | "viewport"
  | "camera"
  | "billboard"
  | "local-plane";
```

Physics normally operates in **world** or **local-plane** space. Screen-space UI should not automatically participate in physical simulation unless explicitly mapped to a simulation plane.

Part II - Time and Motion Architecture

9. Clock and Time Domains

four.js must distinguish multiple time concepts.

```
interface TimeState {  
  realTime: number;  
  renderTime: number;  
  simulationTime: number;  
  deltaTime: number;  
  fixedDeltaTime: number;  
  interpolationAlpha: number;  
  frame: number;  
  simulationStep: number;  
}
```

Required time domains:

- **real time** - wall-clock elapsed time;
- **render time** - time used for visual presentation;
- **simulation time** - deterministic physics time;
- **animation time** - timeline or clip-local time;
- **scaled time** - affected by slow motion or pause;
- **unscaled time** - unaffected by simulation time scale.

```
app.time.scale = 0.25;  
app.time.paused = false;
```

Individual systems may select a time source.

10. Main Loop

The application loop shall separate simulation from rendering.

```
app.on("fixedUpdate", ({ fixedDelta }) => {  
  physics.step(fixedDelta);  
});  
  
app.on("update", ({ delta, alpha }) => {  
  animation.update(delta);  
  controls.update(delta);  
});  
  
app.on("render", () => {  
  renderer.render(scene);  
});
```

Recommended accumulator algorithm:

```
accumulator += elapsedTime;

while (accumulator >= fixedDeltaTime) {
    previousState.copy(currentState);
    simulate(fixedDeltaTime);
    accumulator -= fixedDeltaTime;
}

alpha = accumulator / fixedDeltaTime;
render(interpolate(previousState, currentState, alpha));
```

This design provides:

- stable physics;
 - smooth rendering;
 - deterministic playback;
 - pause and step controls;
 - slow motion;
 - simulation replay.
-

11. Motion Components

A node may use a MotionComponent.

```
interface MotionComponent {
    linearVelocity: Vector3;
    angularVelocity: Vector3;
    linearAcceleration: Vector3;
    angularAcceleration: Vector3;

    damping: number;
    angularDamping: number;
    maxSpeed?: number;
    maxAngularSpeed?: number;
}
```

This component supports non-physics procedural motion and acts as a bridge to physics solvers.

Example:

```
const motion = new Four.Motion({
    linearVelocity: new Vector3(2, 0, 0),
    angularVelocity: new Vector3(0, 1, 0)
});
```

```
node.addComponent(motion);
```

12. Kinematic Motion

Kinematic controllers directly prescribe movement.

```
interface KinematicController {  
    moveTo(position: Vector3, options?: MoveOptions): void;  
    rotateTo(rotation: Quaternion, options?: RotateOptions): void;  
    followPath(path: Curve, options?: PathFollowOptions): void;  
}
```

Required kinematic features:

- velocity-based movement;
 - target following;
 - path following;
 - steering behaviors;
 - look-at constraints;
 - orbit motion;
 - spline motion;
 - camera rigs;
 - character controllers;
 - motion limits.
-

13. Trajectory System

```
interface Trajectory {  
    samplePosition(time: number, out?: Vector3): Vector3;  
    sampleVelocity(time: number, out?: Vector3): Vector3;  
    sampleAcceleration(time: number, out?: Vector3): Vector3;  
    duration: number;  
}
```

Built-in trajectories:

- linear;
- parabolic;
- circular;
- elliptical;
- Bézier;
- Catmull-Rom spline;
- ballistic;
- damped spring;

- custom parametric trajectory.

This is useful for:

- engineering visualization;
 - robotic motion;
 - camera movement;
 - projectile previews;
 - educational simulations;
 - animation paths.
-

Part III - Animation Architecture

14. Animation System Requirements

four.js shall support:

- property animation;
 - keyframe animation;
 - timelines;
 - easing;
 - transform animation;
 - skeletal animation;
 - morph-target animation;
 - material animation;
 - path animation;
 - procedural animation;
 - spring animation;
 - state machines;
 - animation blending;
 - additive animation;
 - inverse kinematics;
 - physics-animation blending.
-

15. Tween API

```
Four.animate(node.position)
  .to({ x: 10, y: 5 }, 1000)
  .ease("cubic-out")
  .play();
```

Required easing families:

- linear;
- quadratic;

- cubic;
 - quartic;
 - quintic;
 - sine;
 - exponential;
 - circular;
 - back;
 - bounce;
 - elastic;
 - spring.
-

16. Timeline API

```
const timeline = new Four.Timeline();

timeline
  .at(0, Four.tween(node.position, { x: 5 }, 800))
  .at(250, Four.tween(node, { opacity: 0.5 }, 500))
  .at(1000, () => console.log("complete"));

timeline.play();
```

Timeline requirements:

- nested timelines;
 - labels;
 - markers;
 - sequencing;
 - parallel tracks;
 - looping;
 - reversing;
 - scrubbing;
 - playback speed;
 - pause and resume;
 - event callbacks;
 - deterministic evaluation.
-

17. Animation Clips and Tracks

```
class AnimationClip {
  name: string;
  duration: number;
  tracks: AnimationTrack[];
```

```
    events: AnimationEvent[];
}
```

Track types:

- scalar;
- vector;
- quaternion;
- color;
- Boolean;
- discrete;
- morph weight;
- skeletal joint;
- custom property.

Interpolation modes:

- step;
 - linear;
 - cubic;
 - Hermite;
 - spherical linear interpolation for quaternions.
-

18. Animation State Machines

```
const controller = new Four.AnimationController({
  states: {
    idle: idleClip,
    walk: walkClip,
    run: runClip
  },
  transitions: [
    { from: "idle", to: "walk", when: "speed > 0.1" },
    { from: "walk", to: "run", when: "speed > 5" }
  ]
});
```

State machine features:

- parameters;
- Boolean conditions;
- numeric comparisons;
- triggers;
- transition duration;
- exit time;
- transition interruption;
- blend trees;

- layered animation.
-

19. Physics-Animation Blending

A node may be:

```
type MotionAuthority =  
  | "animation"  
  | "kinematic"  
  | "physics"  
  | "blended";
```

Examples:

- animated door controlled by a timeline;
- physically simulated door connected by a hinge;
- kinematic robot arm following a commanded path;
- character with animated limbs and physically simulated ragdoll response.

```
body.motionAuthority = "blended";  
body.physicsWeight = 0.35;  
body.animationWeight = 0.65;
```

The implementation must define conflict resolution clearly.

Recommended rule:

1. animation produces a target pose;
 2. kinematic controllers may modify the target;
 3. physics solves constraints and forces;
 4. final rendered pose is interpolated;
 5. optional blending combines animated and physical poses.
-

Part IV - Physics Architecture

20. Physics as a First-Class System

four.js should expose a stable, renderer-independent physics API.

The core framework may use adapter-backed solvers, but users should not need to write solver-specific application code for common tasks.

```
const world = new Four.PhysicsWorld({  
  dimension: "3d",  
  gravity: new Vector3(0, -9.81, 0),  
  solver: "auto"  
});
```

21. Physics Dimensions

```
type PhysicsDimension = "2d" | "3d";
```

The API should be conceptually consistent across both dimensions.

```
const world2D = new Four.PhysicsWorld({  
  dimension: "2d",  
  gravity: new Vector2(0, 9.81)  
});
```

```
const world3D = new Four.PhysicsWorld({  
  dimension: "3d",  
  gravity: new Vector3(0, -9.81, 0)  
});
```

The internal solver may differ, but common operations should use parallel naming and semantics.

22. Body Types

```
type BodyType =  
  | "static"  
  | "dynamic"  
  | "kinematic-position"  
  | "kinematic-velocity";
```

Static

Does not move and is unaffected by forces.

Dynamic

Moves according to mass, force, collisions, and constraints.

Kinematic position

Moves toward prescribed positions.

Kinematic velocity

Moves using prescribed velocity.

23. Rigid Body

```
class RigidBody {
    type: BodyType;

    mass: number;
    inverseMass: number;

    centerOfMass: Vector3;
    inertiaTensor: Matrix3;

    linearVelocity: Vector3;
    angularVelocity: Vector3;

    linearDamping: number;
    angularDamping: number;

    gravityScale: number;
    sleeping: boolean;
    continuousCollisionDetection: boolean;

    applyForce(force: Vector3, point?: Vector3): void;
    applyTorque(torque: Vector3): void;
    applyImpulse(impulse: Vector3, point?: Vector3): void;
    applyAngularImpulse(impulse: Vector3): void;
}
```

24. Collider System

```
interface Collider {
    shape: CollisionShape;
    offset: Transform;
    friction: number;
    restitution: number;
    density: number;
    sensor: boolean;
    collisionGroups: number;
    collisionMask: number;
}
```

Required collision shapes:

2D

- circle;

- rectangle;
- capsule;
- polygon;
- polyline;
- chain;
- compound shape.

3D

- sphere;
- box;
- capsule;
- cylinder;
- cone;
- convex hull;
- triangle mesh;
- height field;
- compound shape.

25. Physics Materials

```
class PhysicsMaterial {
  friction: number;
  restitution: number;
  rollingFriction?: number;
  spinningFriction?: number;
  density: number;
}
```

Combination rules:

```
type CombineMode =
  | "average"
  | "minimum"
  | "maximum"
  | "multiply";
```

26. Forces and Impulses

Required force APIs:

```
body.applyForce(force);
body.applyForceAtPoint(force, worldPoint);
body.applyTorque(torque);
body.applyImpulse(impulse);
```

```
body.applyImpulseAtPoint(impulse, worldPoint);  
body.applyAngularImpulse(angularImpulse);
```

Force generators may include:

- gravity;
 - drag;
 - springs;
 - buoyancy;
 - wind;
 - magnetic approximations;
 - attractors;
 - repulsors;
 - custom fields.
-

27. Force Fields

```
interface ForceField {  
    sample(  
        position: Vector3,  
        velocity: Vector3,  
        time: number  
    ): Vector3;  
}
```

Built-in field types:

- uniform gravity;
- radial gravity;
- vortex;
- wind;
- drag volume;
- turbulence/noise;
- spring field;
- user-defined callback;
- GPU field.

Force fields should support volume-based inclusion and filtering.

28. Constraints and Joints

Required joint types:

Shared concepts

- fixed;

- distance;
- spring;
- revolute/hinge;
- prismatic/slider;
- spherical/ball;
- rope;
- gear;
- motorized joint.

```
const hinge = new Four.HingeJoint({
  bodyA,
  bodyB,
  anchor,
  axis,
  limits: {
    min: -Math.PI / 2,
    max: Math.PI / 2
  },
  motor: {
    enabled: true,
    targetVelocity: 2,
    maxTorque: 50
  }
});
```

Constraint features:

- limits;
- motors;
- springs;
- damping;
- break force;
- break torque;
- collision enable/disable;
- solver iterations.

29. Collision Events

```
body.on("collisionstart", event => {});
body.on("collisionstay", event => {});
body.on("collisionend", event => {});

sensor.on("triggerenter", event => {});
sensor.on("triggerexit", event => {});
```

Collision event data:

```

interface CollisionEvent {
  bodyA: RigidBody;
  bodyB: RigidBody;
  colliderA: Collider;
  colliderB: Collider;
  contacts: ContactPoint[];
  relativeVelocity: Vector3;
  totalImpulse: Vector3;
}

```

30. Queries

Required physics queries:

```

world.raycast(ray, options);
world.shapeCast(shape, transform, direction, options);
world.overlapSphere(center, radius, options);
world.overlapBox(center, halfExtents, rotation, options);
world.pointQuery(point, options);

```

Queries should support:

- collision groups;
 - masks;
 - ignored bodies;
 - first hit;
 - all hits;
 - sorted hits;
 - sensor inclusion;
 - custom filters.
-

31. Continuous Collision Detection

Fast objects may tunnel through thin geometry. four.js shall provide optional continuous collision detection.

```
body.continuousCollisionDetection = true;
```

Possible modes:

```

type CCDMode =
  | "disabled"
  | "speculative"
  | "swept";

```

32. Sleeping

Dynamic bodies at rest should sleep to improve performance.

```
world.sleeping = {  
  enabled: true,  
  linearThreshold: 0.01,  
  angularThreshold: 0.01,  
  timeThreshold: 0.5  
};
```

Users should be able to wake bodies explicitly.

33. Determinism

four.js should define determinism tiers.

```
type DeterminismLevel =  
  | "none"  
  | "same-runtime"  
  | "same-platform"  
  | "cross-platform";
```

The initial target should be **same-runtime deterministic simulation** when:

- the same solver is used;
- the same timestep is used;
- the same input sequence is used;
- multithreaded nondeterministic paths are disabled.

The engine should support:

- seeded random number generators;
 - recorded inputs;
 - state snapshots;
 - replay;
 - rollback;
 - checksums.
-

34. Physics Snapshots and Replay

```
const snapshot = world.createSnapshot();
```

```
world.restoreSnapshot(snapshot);
```

Use cases:

- debugging;

- network rollback;
- deterministic tests;
- simulation comparison;
- education;
- engineering analysis.

A replay format should store:

- initial scene state;
- solver settings;
- time step;
- random seed;
- external inputs;
- optional periodic snapshots.

35. Soft Bodies and Deformables

Soft-body support should be a later module.

Potential features:

- cloth;
- ropes;
- deformable surfaces;
- volume preservation;
- position-based dynamics;
- mass-spring systems;
- shape matching.

@four/physics-soft

This should not block the core rigid-body MVP.

36. Particles

Particles should support both visual-only and physically simulated modes.

```
const emitter = new Four.ParticleEmitter({
  maxParticles: 100000,
  simulation: "gpu",
  forces: [gravity, wind],
  collisions: "depth-buffer"
});
```

Particle features:

- CPU simulation;

- GPU compute simulation;
- emitters;
- lifetimes;
- velocity distributions;
- forces;
- color and size over lifetime;
- collision;
- trails;
- attractors;
- custom data channels.

37. Physics Solver Adapter

```
interface PhysicsSolverAdapter {
  readonly name: string;
  readonly capabilities: PhysicsCapabilities;

  initialize(options: PhysicsWorldOptions): Promise<void> | void;
  createBody(desc: RigidBodyDescriptor): PhysicsBodyHandle;
  createCollider(desc: ColliderDescriptor): PhysicsColliderHandle;
  createJoint(desc: JointDescriptor): PhysicsJointHandle;

  step(delta: number): void;
  syncToScene(): void;
  syncFromScene(): void;

  raycast(query: RaycastQuery): RaycastHit[];
  createSnapshot?(): ArrayBuffer;
  restoreSnapshot?(snapshot: ArrayBuffer): void;

  dispose(): void;
}
```

Potential adapters:

- Rapier 2D/3D;
- Box2D;
- Matter.js;
- Cannon-es;
- Ammo.js;
- custom engineering solvers.

The stable four.js API should sit above these adapters.

Part V - Numerical Integration and Simulation

38. Integrators

For built-in lightweight motion, four.js should provide:

```
type Integrator =  
  | "explicit-euler"  
  | "semi-implicit-euler"  
  | "velocity-verlet"  
  | "rk2"  
  | "rk4";
```

Recommended defaults:

- semi-implicit Euler for simple real-time rigid motion;
- velocity Verlet for conservative particle systems;
- RK4 for small, smooth engineering demonstrations where accuracy matters more than cost.

The full rigid-body solver adapter may use its own integration method.

39. Simulation Systems

```
interface SimulationSystem {  
  priority: number;  
  initialize(context: SimulationContext): void;  
  fixedUpdate(context: FixedUpdateContext): void;  
  dispose(): void;  
}
```

Example system order:

1. Input sampling
2. Command processing
3. Animation target evaluation
4. Kinematic motion
5. Force generation
6. Physics solve
7. Constraint solve
8. Collision event dispatch
9. Sensor update
10. State snapshot
11. Render interpolation

The ordering must be explicit and configurable.

40. Units

four.js should not silently assume that one world unit is always one meter.

```
interface UnitSystem {
  length: "meter" | "centimeter" | "millimeter" | "custom";
  mass: "kilogram" | "gram" | "custom";
  time: "second" | "millisecond";
  angle: "radian" | "degree";
  scale: {
    lengthToMeters: number;
    massToKilograms: number;
  };
}
```

Recommended physics default:

```
length = meter
mass    = kilogram
time    = second
angle   = radian
```

Engineering applications must be able to declare and display units explicitly.

41. Numerical Stability Guidance

The engine documentation should explain:

- why very large mass ratios can destabilize solvers;
- why extremely small or large world scales are problematic;
- why fixed timesteps are preferred;
- how solver iterations affect stability;
- how collision margins work;
- how damping differs from friction;
- how continuous collision detection affects performance.

The diagnostics package should warn about suspicious values.

Part VI - Rendering and Motion Synchronization

42. Transform Authority

A transform may be controlled by:

```
type TransformAuthority =
  | "manual"
```

```
| "animation"  
| "kinematic"  
| "physics"  
| "constraint"  
| "network";
```

The engine must prevent multiple systems from silently overwriting the same transform.

```
node.transformAuthority = "physics";
```

Conflicts should produce development warnings.

43. Physics-to-Render Synchronization

Physics state updates at fixed intervals. Rendering may occur at a different rate.

```
renderPosition = lerp(  
    previousPhysicsPosition,  
    currentPhysicsPosition,  
    interpolationAlpha  
);
```

Rotations should use quaternion spherical interpolation.

The render transform should not feed back into the physics state unless explicitly requested.

44. Camera Motion

Cameras should support:

- orbit control;
- fly control;
- first-person control;
- trackball control;
- follow rigs;
- spring arms;
- shake;
- path animation;
- physics attachment.

Camera motion should use the same timeline, constraint, and motion systems as ordinary nodes.

Part VII - Complete Graphics, Rendering, Application, and Platform Architecture

45. Application Model

The high-level `Application` object owns the default scene, renderer, time system, simulation scheduler, input routing, assets, diagnostics, cameras, and viewports.

```
const app = new Four.Application({
  canvas: document.querySelector("canvas"),
  renderer: "auto",
  antialias: true,
  resolution: window.devicePixelRatio,
  fixedTimeStep: 1 / 60,
  physics: { solver: "rapier", dimension: "3d" }
});

await app.initialize();
app.start();

interface ApplicationOptions {
  canvas?: HTMLCanvasElement | OffscreenCanvas;
  renderer?: "auto" | "webgpu" | "webgl2" | "canvas2d" | "svg";
  width?: number;
  height?: number;
  resolution?: number;
  antialias?: boolean;
  alpha?: boolean;
  powerPreference?: "low-power" | "high-performance";
  autoResize?: boolean;
  fixedTimeStep?: number;
  maximumSubSteps?: number;
  physics?: PhysicsWorldOptions | false;
}
```

The application lifecycle shall expose:

- `initialize`;
- `start`;
- `stop`;
- `pause`;
- `resume`;
- `step`;
- `resize`;
- `dispose`.

The application must permit advanced users to construct and own these systems

independently rather than requiring the convenience wrapper.

46. Scene Queries, Layers, and Tags

A `Scene` provides indexed lookup by identifier, name, type, tag, component, and optional selector syntax.

```
scene.findById("motor-01");
scene.findByName("bearing");
scene.findByTag("sensor");
scene.findByComponent(RigidBody);
scene.query("Mesh.dynamic[visible=true]");
```

Symbolic layers control:

- camera visibility;
- rendering order;
- physics interaction groups;
- picking and pointer interaction;
- post-processing inclusion;
- editor-only objects;
- debug visualization.

Layers should compile to efficient masks internally while preserving human-readable names in the public API and serialized scene files.

47. Camera System

Required camera types:

- `PerspectiveCamera`;
- `OrthographicCamera`;
- `ScreenCamera`;
- `ObliqueCamera`;
- custom projection camera.

```
abstract class Camera extends Node {
  near: number;
  far: number;
  projectionMatrix: Matrix4;
  inverseProjectionMatrix: Matrix4;
  viewMatrix: Matrix4;
  layers: LayerMask;
}
```

`ScreenCamera` shall support top-left, bottom-left, and centered origins with logical-pixel or physical-pixel units.

Camera rigs shall include:

- orbit;
- fly;
- first-person;
- trackball;
- follow target;
- spring arm;
- stereo/XR extension point;
- shake and impulse effects.

48. Viewports and Render Surfaces

A viewport maps one camera to a rectangular region and optional render target.

```
interface Viewport {
  id: string;
  camera: Camera;
  normalized?: boolean;
  x: number;
  y: number;
  width: number;
  height: number;
  clearColor?: Color;
  clearDepth?: number;
  layerMask?: LayerMask;
  renderTarget?: RenderTarget;
  postProcessing?: RenderPipeline;
}
```

Supported use cases:

- split-screen;
- minimaps;
- CAD orthographic views;
- picture-in-picture;
- editor panels;
- offscreen textures;
- mirrors and portals;
- 3D model previews inside 2D UI.

49. Renderable Node Hierarchy

```
Renderable
+-- Shape2D
|   +-- Circle
|   +-- Ellipse
|   +-- Rectangle
|   +-- RoundedRectangle
```

```

|   +-- Polygon
|   +-- Polyline
|   +-- Arc
|   +-- Path
+-- Sprite
+-- Text
+-- Mesh
+-- Line3D
+-- PointCloud
+-- ParticleSystem
+-- CustomRenderable

abstract class Renderable extends Node {
  material: Material | Material[];
  renderLayer: number;
  renderOrder: number;
  depthMode: "normal" | "always-front" | "always-back" | "disabled";
  castShadow: boolean;
  receiveShadow: boolean;
  frustumCulled: boolean;
}

```

50. Native 2D Shape System

Required shape primitives:

- circle;
- ellipse;
- rectangle;
- rounded rectangle;
- regular polygon;
- arbitrary polygon;
- star;
- line;
- polyline;
- arc;
- sector;
- ring;
- path;
- Bézier path.

```

const rectangle = new Four.Rectangle({
  width: 200,
  height: 100,
  radius: 12,
  fill: "#4466ff",
  stroke: { color: "ffffff", width: 3 }
}

```

```
});
```

Shape requirements:

- fill and stroke;
- fill opacity and stroke opacity;
- stroke alignment;
- dashes and dash offset;
- miter, bevel, and round joins;
- butt, square, and round caps;
- clipping and masks;
- Boolean geometry operations;
- local and world bounds;
- analytic hit testing where possible;
- SVG import/export compatibility.

51. Path Model

```
const path = new Four.Path();
path.moveTo(0, 0);
path.lineTo(100, 0);
path.quadraticCurveTo(150, 50, 100, 100);
path.cubicCurveTo(75, 125, 25, 125, 0, 100);
path.arc(0, 50, 25, 0, Math.PI);
path.close();
```

Required operations:

- move;
- line;
- quadratic Bézier;
- cubic Bézier;
- circular and elliptical arc;
- close path;
- flatten;
- subdivide;
- simplify;
- reverse;
- transform;
- compute length;
- evaluate point, tangent, and normal;
- closest-point query;
- offset path;
- union, intersection, subtraction, and xor.

Fill rules:

- nonzero;
- even-odd.

52. Tessellation and Stroke Generation

2D paths must be converted into GPU-ready geometry while retaining vector-level source data.

The tessellation subsystem shall support:

- concave polygons;
- holes;
- self-intersections where well-defined;
- adaptive curve subdivision;
- stroke expansion;
- anti-alias fringe generation;
- index-buffer reuse;
- incremental rebuild of modified path segments;
- optional compute-based tessellation in later releases.

The tessellator shall be an isolated package with a stable interface so implementations can be replaced without changing the scene API.

53. Geometry Architecture

```
abstract class Geometry implements Disposable {
    readonly id: string;
    version: number;
    bounds: BoundingBox;
    computeBounds(): void;
    clone(): Geometry;
    dispose(): void;
}
```

```
Geometry
+-- Geometry2D
|   +-- PathGeometry2D
|   +-- FillGeometry2D
|   +-- StrokeGeometry2D
+-- Geometry3D
    +-- BufferGeometry
    +-- IndexedGeometry
    +-- ProceduralGeometry
```

Required 3D primitives:

- plane;
- box;
- sphere;
- cylinder;
- cone;
- capsule;

- torus;
- lathe;
- extrusion;
- tube;
- height field.

Standard attributes:

- position;
- normal;
- tangent;
- color;
- uv and secondary uv;
- joints and weights;
- instance transform;
- custom typed attributes.

54. Mesh, Instancing, and Level of Detail

```
class Mesh extends Renderable {
  geometry: Geometry3D;
  material: Material | Material[];
  morphTargetWeights?: Float32Array;
  skeleton?: Skeleton;
}
```

The engine shall support:

- indexed and non-indexed geometry;
- multiple material groups;
- hardware instancing;
- indirect rendering where supported;
- morph targets;
- skeletal deformation;
- static and dynamic GPU buffers;
- level-of-detail selection;
- impostors and billboards;
- geometry merging and batching tools.

55. Sprite and Raster System

Sprites shall support:

- screen-space and world-space sizing;
- anchors and pivots;
- atlases and frame regions;
- nine-slice scaling;
- tint and opacity;

- billboard modes;
- per-instance data;
- alpha masks;
- normal-mapped sprites as an extension;
- sprite animation clips.

```
class Sprite extends Renderable {
  texture: Texture;
  frame?: Rectangle2;
  anchor: Vector2;
  sizeMode: "pixels" | "world";
  billboardMode: "none" | "spherical" | "cylindrical";
}
```

56. Text and Typography

Text is a core capability rather than a UI-only afterthought.

Requirements:

- Unicode;
- font fallback;
- bidirectional layout;
- ligatures;
- kerning;
- shaping;
- line breaking and wrapping;
- horizontal and vertical alignment;
- letter and word spacing;
- rich text spans;
- text along paths;
- world-space, billboard, and screen-space text;
- bitmap, signed-distance-field, and multi-channel SDF rendering;
- selection and caret support in UI text inputs;
- accessible semantic mirror.

```
const label = new Four.Text({
  text: "Motor Temperature",
  fontFamily: "Inter",
  fontSize: 22,
  fontWeight: 600,
  color: "#ffffff",
  space: "billboard"
});
```

57. Unified Material Model

Shared material properties:


```

abstract class Material implements Disposable {
    opacity: number;
    transparent: boolean;
    blendMode: BlendMode;
    depthTest: boolean;
    depthWrite: boolean;
    colorWrite: boolean;
    stencil?: StencilState;
    dispose(): void;
}

```

Material families:

```

Material
+-- ShapeMaterial
+-- SpriteMaterial
+-- TextMaterial
+-- LineMaterial
+-- UnlitMaterial
+-- StandardMaterial
+-- PhysicalMaterial
+-- ShaderMaterial
+-- NodeMaterial
+-- ComputeMaterial

```

The API unifies lifecycle and render state while preserving specialized 2D and 3D properties.

58. Paints, Fills, and Strokes

A shape paint may be:

- solid color;
- linear gradient;
- radial gradient;
- conic gradient;
- image pattern;
- procedural shader;
- render-target texture.

```

interface StrokeStyle {
    paint: Paint;
    width: number;
    alignment: "inside" | "center" | "outside";
    lineCap: "butt" | "round" | "square";
    lineJoin: "miter" | "round" | "bevel";
    miterLimit: number;
    dash?: number[];
}

```

```
dashOffset?: number;
}
```

59. Physically Based Materials

StandardMaterial shall implement a metallic-roughness workflow compatible with glTF conventions.

```
const material = new Four.StandardMaterial({
  baseColor: "#a0a0a0",
  roughness: 0.6,
  metalness: 0.1,
  normalMap,
  occlusionMap,
  emissive: "#000000"
});
```

Later physical extensions may include:

- clearcoat;
- transmission;
- index of refraction;
- sheen;
- anisotropy;
- subsurface approximation;
- iridescence.

60. Shader and Node-Material System

Advanced users require a backend-independent shader model.

```
const material = new Four.NodeMaterial();
const albedo = material.texture(albedoTexture);
const pulse = material.sin(material.time().multiply(2));
material.output.color = albedo.multiply(pulse.add(1));
```

The compiler should generate:

- WGSL for WebGPU;
- GLSL ES for WebGL 2;
- reduced fallbacks for Canvas/SVG where meaningful.

Shader features:

- reusable functions;
- uniforms and uniform blocks;
- vertex attributes;
- storage buffers;
- textures and samplers;
- conditional variants;

- reflection metadata;
- source maps and readable compiler diagnostics.

61. Renderer Interface

```
interface Renderer {
  readonly capabilities: RendererCapabilities;
  initialize(options: RendererOptions): Promise<void>;
  render(scene: Scene, views: readonly Viewport[]): void;
  resize(width: number, height: number, resolution: number): void;
  createTexture(source: TextureSource): Texture;
  createRenderTarget(options: RenderTargetOptions): RenderTarget;
  readPixels?(target: RenderTarget, region?: Rectangle2): Promise<ArrayBuffer>;
  dispose(): void;
}
```

The logical scene shall remain independent of the selected backend.

62. Rendering Backends and Capability Tiers

Supported backends:

1. WebGPU;
2. WebGL 2;
3. Canvas 2D;
4. SVG;
5. headless/software extension.

```
renderer: "auto"
```

Automatic selection should prefer WebGPU, then WebGL 2, then an appropriate 2D backend.

Capability reporting shall include:

- maximum texture dimensions;
- texture formats;
- multisampling;
- floating-point targets;
- timestamp queries;
- storage buffers;
- compute shaders;
- indirect draw;
- compressed textures;
- shader precision;
- maximum uniforms and bindings.

Applications may declare required and optional capabilities.

63. Render Graph

Rendering shall be organized as a directed acyclic graph of passes and resources.

Scene Preparation

v

Depth Prepass (optional)

v

Shadow Passes

v

Opaque World

v

Transparent World

v

World-Space Vectors and Text

v

Post-Processing

v

Screen-Space UI

v

Final Composite

```
const graph = new Four.RenderGraph();
graph.addPass("world", worldPass);
graph.addPass("bloom", bloomPass, { inputs: ["world"] });
graph.addPass("ui", uiPass);
graph.addPass("composite", compositePass, { inputs: ["bloom", "ui"] });
```

The graph shall manage:

- pass dependencies;
- transient render targets;
- resource lifetime;
- barriers and state transitions;
- pass enable/disable;
- viewport-specific pipelines;
- debug visualization.

64. Render Preparation and Submission

The renderer pipeline shall separate:

1. scene traversal;
2. visibility and layer filtering;
3. frustum and occlusion culling;
4. render-item generation;
5. sorting;
6. batching and instancing;
7. backend command encoding;

8. GPU submission.

Avoid per-node virtual calls in the final drawing hot path. Renderables should compile into compact render items and backend-native pipelines.

65. Batching and Instancing

Automatic batching strategies:

- sprite batching;
- glyph batching;
- compatible shape batching;
- instanced meshes;
- material sorting;
- pipeline sorting;
- texture atlas grouping;
- persistent mapped or staged buffers;
- multi-draw/indirect draw where available.

Batching shall be transparent to ordinary users but inspectable through diagnostics.

66. Ordering, Transparency, and Composition

Default sorting order:

1. render layer;
2. opaque versus transparent classification;
3. pipeline/material compatibility;
4. depth;
5. explicit render order.

The engine must document limitations of transparent sorting and provide:

- order-independent transparency extension points;
- weighted blended transparency option;
- depth prepass control;
- explicit object ordering;
- alpha test and alpha-to-coverage;
- premultiplied and straight alpha policies.

Screen-space UI normally renders after world content. World-space 2D geometry normally participates in depth testing.

67. Clipping, Masks, and Stencils

Required mechanisms:

- rectangular scissor clipping;
- path masks;

- alpha masks;
- stencil masks;
- nested clipping;
- UI overflow clipping;
- clipping planes for 3D;
- section views for engineering models.

Nested clipping must have defined behavior and diagnostics when backend limits are exceeded.

68. Lighting

Initial lights:

- ambient;
- hemisphere;
- directional;
- point;
- spot;
- rectangular area light where supported.

```
const light = new Four.DirectionalLight({
  color: "#ffffff",
  intensity: 3,
  castShadow: true
});
```

Lighting requirements:

- physically coherent units where practical;
- light layers;
- environment lighting;
- image-based lighting;
- tone mapping;
- exposure;
- clustered/forward-plus extension path for many lights.

69. Shadows

Required shadow features:

- directional shadow maps;
- point-light cubemap shadows;
- spot-light shadows;
- cascaded shadow maps;
- configurable resolution;
- bias and normal-bias controls;
- percentage-closer filtering;
- transparent shadow masks;

- contact-shadow extension;
- shadow atlas management.

70. Post-Processing

The render graph shall support reusable effects:

- tone mapping;
- color grading;
- bloom;
- anti-aliasing;
- depth of field;
- motion blur;
- screen-space ambient occlusion;
- outlines and selection highlighting;
- distortion;
- custom full-screen passes.

Effects must be composable per viewport.

71. Picking and Hit Testing

One unified picking API shall cover 2D and 3D.

Strategies:

- analytic primitive testing;
- bounding-volume testing;
- path geometry testing;
- ray/triangle intersection;
- pixel-alpha testing;
- GPU identifier buffer;
- custom callbacks.

```
node.hitTestMode = "bounds" | "geometry" | "pixel" | "gpu" | "custom";
```

The engine should select the cheapest valid method by default.

72. Input and Event Propagation

Input sources:

- mouse;
- touch;
- pen/stylus;
- keyboard;
- wheel and trackpad;
- gamepad;
- future XR controllers.

Event phases mirror the DOM:

Capture -> **Target** -> **Bubble**

Events include pointer enter/leave, down/up/move, click, double-click, wheel, drag, pinch, rotate, keyboard, focus, and blur.

Pointer capture must be supported across mixed 2D/3D objects.

73. Retained-Mode UI

The optional `@four/ui` package shall provide:

- panel;
- label;
- button;
- toggle;
- checkbox;
- radio control;
- slider;
- text input;
- scroll view;
- list and virtual list;
- image;
- progress indicator;
- menu;
- tooltip;
- canvas view;
- embedded 3D viewport.

UI objects are scene nodes and therefore share animation, input, clipping, serialization, and diagnostics.

74. Layout

Required layout modes:

- absolute;
- stack;
- flex;
- grid;
- anchor;
- constraints.

```
const panel = new Four.Panel({
  layout: {
    type: "flex",
    direction: "column",
    gap: 12,
```



```

        padding: 20
    }
});

```

Layout must support:

- logical pixels;
- percentages;
- minimum/maximum sizes;
- intrinsic text/image size;
- margins and padding;
- overflow;
- scroll extent;
- device-pixel scaling;
- right-to-left interfaces.

75. Accessibility

The UI module shall provide an optional hidden DOM accessibility mirror.

```

button.accessibility = {
  role: "button",
  label: "Start simulation",
  description: "Begins the motor simulation",
  tabIndex: 0
};

```

Requirements:

- semantic roles;
- accessible names and descriptions;
- keyboard navigation;
- focus management;
- disabled/checked/expanded states;
- screen-reader updates;
- reduced-motion preference;
- high-contrast theme hooks;
- scalable text.

76. Asset System

```

const assets = await app.assets.load({
  robot: "/models/robot.glb",
  icon: "/images/icon.png",
  font: "/fonts/inter.woff2"
});

```

Supported initial formats:

- PNG, JPEG, WebP, and AVIF where available;
- SVG;
- JSON;
- glTF and GLB;
- font files;
- audio files through optional module;
- OBJ and other legacy formats through plugins.

The asset manager shall support:

- deduplication;
- caching;
- reference counting;
- lazy loading;
- streaming;
- dependency graphs;
- progress reporting;
- cancellation;
- retries;
- worker decoding;
- hot reload in development;
- content hashing.

77. Texture System

Texture requirements:

- 2D, cube, array, and 3D textures where supported;
- mipmaps;
- wrap and filter modes;
- anisotropy;
- color-space metadata;
- compressed texture containers;
- render-target textures;
- video textures;
- canvas and image-bitmap sources;
- asynchronous upload and residency diagnostics.

78. Model and Scene Loading

The glTF loader should support:

- geometry;
- materials;
- textures;
- skins;
- morph targets;
- animations;

- cameras;
- lights extensions;
- compression extensions through optional decoders;
- user metadata.

Loaded assets should be instantiated without sharing mutable transforms while safely sharing immutable geometry and textures.

79. Serialization and Scene Format

Human-readable scene files use:

`.four.json`

Binary packages may use:

`.four`

Serialization goals:

- versioned;
- deterministic;
- backend-independent;
- schema-validatable;
- diff-friendly;
- extensible;
- capable of preserving unknown extension data.

```
{
  "format": "four-scene",
  "version": "1.0",
  "scene": {
    "type": "Scene",
    "id": "scene-main",
    "children": []
  }
}
```

Physics state, animation state, and replay data must be separate optional sections so static scene definitions remain clean.

80. Scene Migration

Scene format versioning is independent from package semantic versioning.

```
const migrated = Four.SceneMigrator.upgrade(oldScene, "2.0");
```

Migrations must be:

- explicit;
- testable;

- deterministic;
- capable of producing warnings for lossy changes;
- composable across multiple versions.

81. Plugin System

```
interface FourPlugin {
  name: string;
  version: string;
  install(context: PluginContext): void | Promise<void>;
  uninstall?(context: PluginContext): void;
}
```

Plugin extension points:

- render passes;
- renderer backends;
- asset formats;
- materials and shader nodes;
- physics solvers;
- animation systems;
- UI controls;
- editor tools;
- diagnostics;
- serialization types;
- compute workloads.

Plugins shall declare dependencies and compatibility ranges.

82. GPU Compute

WebGPU compute is an advanced optional capability for:

- particles;
- image processing;
- procedural geometry;
- physics preprocessing;
- cellular automata;
- field simulation;
- matrix operations;
- scientific visualization.

```
const compute = new Four.ComputePass({
  shader: particleShader,
  workgroups: [1024, 1, 1],
  bindings: { positions, velocities, parameters }
});
```

Basic graphics and physics functionality must not require compute support.

83. Resource Lifecycle

GPU and solver resources shall be explicitly disposable.

```
texture.dispose();  
geometry.dispose();  
material.dispose();  
physicsWorld.dispose();  
app.dispose();
```

The engine should also implement reference counting or ownership tracking for shared resources.

Development warnings:

- leaked textures/buffers;
- disposed resources still in use;
- duplicate asset loads;
- detached nodes retaining listeners;
- stale physics handles;
- excessive per-frame allocations.

84. Diagnostics and Developer Tools

Runtime statistics:

```
app.stats.cpuFrameTime;  
app.stats.gpuFrameTime;  
app.stats.simulationTime;  
app.stats.physicsStepTime;  
app.stats.drawCalls;  
app.stats.triangles;  
app.stats.instances;  
app.stats.activeBodies;  
app.stats.contacts;  
app.stats.textureMemory;  
app.stats.bufferMemory;
```

Debug overlays:

- bounds;
- transforms and pivots;
- camera frustums;
- light volumes;
- colliders;
- contacts and normals;
- centers of mass;
- velocity, acceleration, force, and torque vectors;
- joints and limits;
- sleeping bodies;

- overdraw;
- batch boundaries;
- texture atlases;
- render graph;
- UI layout boxes;
- picking identifiers.

85. Validation

Development builds shall detect:

- NaN and infinite values;
- singular transforms;
- scene graph cycles;
- invalid geometry indices;
- unsupported renderer features;
- shader compilation failures;
- conflicting transform authority;
- invalid physics dimensions;
- impossible mass/inertia values;
- unstable scales and extreme ratios;
- serialization version mismatches.

Production builds may disable expensive validation while preserving essential safety checks.

86. Performance Targets

Initial engineering targets on suitable modern desktop hardware:

Scenario	Target
Batched sprites	100,000 at 60 FPS
Simple batched shapes	50,000 at 60 FPS
Simple mesh instances	100,000 visible instances where GPU-bound limits permit
Retained UI nodes	5,000
Animated glyphs	20,000
CPU particles	25,000 baseline
GPU particles	100,000+
Active rigid bodies	5,000 simple bodies baseline
Idle scene	Near-zero unnecessary uploads and simulation work

Targets are benchmark goals, not universal guarantees.

87. Spatial Indexing and Culling

Potential spatial structures:

- dynamic AABB tree;
- quadtree;
- octree;
- bounding-volume hierarchy;
- grid/spatial hash;
- backend-provided physics broad phase.

Systems may maintain specialized indices for rendering, picking, physics, and UI. The public scene graph must not be forced to mirror a spatial tree.

88. Threading and Workers

Operating modes:

Main-thread mode

Input, simulation, and rendering occur on the browser main thread.

Worker-rendering mode

The main thread owns DOM, accessibility, and input forwarding. A worker owns the scene, simulation, and `OffscreenCanvas` rendering.

Split-simulation mode

Rendering stays on the main thread while simulation executes in a worker using transferable or shared state buffers.

The MVP may begin on the main thread, but APIs and data structures should avoid assumptions that make worker migration impossible.

89. Error Model

```
class FourError extends Error {  
  code: string;  
  context?: Record<string, unknown>;  
  cause?: unknown;  
}
```

Example codes:

- `RENDERER_INITIALIZATION_FAILED`;
- `UNSUPPORTED_GPU_FEATURE`;
- `ASSET_LOAD_FAILED`;
- `SHADER_COMPILATION_FAILED`;
- `INVALID_SCENE_GRAPH`;

- PHYSICS_SOLVER_FAILED;
- SERIALIZATION_VERSION_MISMATCH.

Recoverable failures should be reportable through events and diagnostics without always terminating the application.

90. Versioning and Compatibility

The packages shall use semantic versioning.

- patch: compatible defect correction;
- minor: backward-compatible feature;
- major: breaking API change.

The project should publish compatibility tables for:

- browser support;
- WebGPU/WebGL feature tiers;
- physics solver adapters;
- scene format versions;
- plugin API versions.

91. Coding Standards and Toolchain

Recommended baseline:

- strict TypeScript;
- ESM;
- pnpm workspace;
- Turborepo or Nx;
- Vitest;
- Playwright;
- ESLint;
- Prettier;
- API Extractor or TypeDoc;
- Vite;
- Changesets;
- GitHub Actions.

Requirements:

- no implicit `any`;
- documented public APIs;
- tree-shakable modules;
- package-boundary checks;
- unit, integration, visual, and benchmark tests;
- browser compatibility matrix;
- changelog for public releases.

92. Testing Strategy

Unit tests

- vectors, matrices, and quaternions;
- transforms;
- scene graph;
- clocks and scheduling;
- animation interpolation;
- geometry generation;
- path operations;
- serialization;
- physics descriptors and adapter normalization.

Integration tests

- scene plus renderer;
- fixed-step physics plus interpolated rendering;
- picking across 2D and 3D;
- asset loading plus materials;
- animation-to-physics transitions;
- UI focus and accessibility bridge.

Visual regression tests

- shape fills and strokes;
- path joins and caps;
- transparency;
- materials and lighting;
- text layout;
- clipping;
- mixed 2D/3D ordering;
- debug overlays.

Determinism tests

- identical input stream produces identical checksums;
- snapshot restoration reproduces subsequent states;
- replay remains stable within the declared determinism tier.

Performance tests

Track CPU time, GPU time, simulation time, draw calls, contacts, memory, allocations, and loading throughput.

93. Documentation Plan

Documentation shall include:

- installation and quick start;
- first 2D scene;
- first 3D scene;
- first animated scene;
- first physics scene;
- mixed 2D/3D/physics example;
- scene graph and transforms;
- cameras and coordinate conversion;
- materials and render graph;
- motion authority;
- fixed-step simulation;
- collision filtering;
- units and numerical stability;
- performance optimization;
- custom shaders;
- custom solver adapters;
- engineering dashboard guide;
- digital-twin guide.

API documentation should be generated directly from TypeScript declarations, and every major feature should have a runnable example.

94. Release Strategy

- **0.1:** math, scene, time, and basic WebGL rendering;
- **0.2:** native 2D shapes, sprites, text, and picking;
- **0.3:** 3D meshes, materials, lights, shadows, and mixed scenes;
- **0.4:** motion, tweens, timelines, and path animation;
- **0.5:** first physics adapter, bodies, colliders, forces, and collision events;
- **0.6:** joints, motors, animation-physics blending, and replay;
- **0.7:** assets, glTF, serialization, UI, and accessibility;
- **0.8:** WebGPU preview, render graph, compute particles, and workers;
- **0.9:** optimization, conformance, API stabilization, and production trials;
- **1.0:** stable API, stable scene format, compatibility policy, full documentation.

95. Governance

Initial governance may use a lead-maintainer model with delegated ownership for:

- core and math;
- rendering;
- motion and animation;
- physics;
- UI and accessibility;
- documentation;

- release engineering.

Major architectural changes require an RFC or ADR containing:

1. context;
2. proposed decision;
3. alternatives;
4. consequences;
5. compatibility analysis;
6. prototype or benchmark where practical;
7. maintainer approval.

96. Security and Untrusted Content

Asset loaders and scene deserializers shall treat external content as untrusted.

Requirements:

- bounds checking;
- input-size limits;
- decompression limits;
- no arbitrary code execution from scene files;
- safe shader/plugin boundaries;
- cancellation and timeouts for expensive decoders;
- documented content-security-policy behavior.

97. Complete Mixed-Scene Example

```
import * as Four from "four";

const app = new Four.Application({
  canvas: document.querySelector("#app"),
  renderer: "auto",
  fixedTimeStep: 1 / 60,
  physics: { dimension: "3d", gravity: [0, -9.81, 0] }
});

await app.initialize();

const camera = new Four.PerspectiveCamera({
  fieldOfView: 60,
  near: 0.1,
  far: 1000
});
camera.position.set(0, 2, 6);
app.scene.activeCamera = camera;

const cube = new Four.Mesh({
```

```

    geometry: new Four.BoxGeometry({ width: 1, height: 1, depth: 1 }),
    material: new Four.StandardMaterial({ baseColor: "#5588ff" })
  });

cube.addComponent(new Four.RigidBody({ type: "dynamic", mass: 1 }));
cube.addComponent(new Four.BoxCollider({ size: [1, 1, 1] }));

const label = new Four.Text({
  text: "Transformer T-101",
  fontSize: 24,
  color: "ffffff",
  space: "billboard"
});
label.position.set(0, 1.2, 0);
cube.add(label);

const panel = new Four.Panel({
  space: "screen",
  width: 320,
  height: 180,
  x: 20,
  y: 20,
  layout: { type: "flex", direction: "column", gap: 8 }
});

const impulseButton = new Four.Button({ label: "Apply Impulse" });
impulseButton.on("click", () => {
  cube.getComponent(Four.RigidBody)?.applyImpulse(new Four.Vector3(0, 4, 0));
});

panel.add(new Four.Text({ text: "System Status" }), impulseButton);
app.scene.add(cube, panel);

app.start();

```

Part VII - Package Architecture

45. Proposed Monorepo

```

four.js/
+-- packages/
|   +-- core/
|   +-- math/
|   +-- scene/
|   +-- motion/

```

```
| +-- animation/  
| +-- physics/  
| +-- physics-rapier/  
| +-- physics-box2d/  
| +-- physics-soft/  
| +-- particles/  
| +-- geometry/  
| +-- materials/  
| +-- render/  
| +-- render-webgpu/  
| +-- render-webgl/  
| +-- render-canvas/  
| +-- render-svg/  
| +-- input/  
| +-- assets/  
| +-- text/  
| +-- ui/  
| +-- serialization/  
| +-- diagnostics/  
| +-- four/  
+-- examples/  
+-- benchmarks/  
+-- docs/  
+-- tests/  
+-- tools/  
+-- website/
```

46. Motion Package

@four/motion responsibilities:

- clocks;
 - fixed-step scheduler;
 - motion components;
 - velocity and acceleration;
 - kinematic controllers;
 - path following;
 - trajectories;
 - spring motion;
 - steering;
 - interpolation;
 - transform authority.
-

47. Animation Package

@four/animation responsibilities:

- tweens;
 - easing;
 - timelines;
 - clips;
 - tracks;
 - state machines;
 - blend trees;
 - skeletons;
 - inverse kinematics;
 - physics-animation blending.
-

48. Physics Package

@four/physics responsibilities:

- stable public API;
 - body and collider descriptors;
 - physics materials;
 - constraints;
 - joints;
 - force fields;
 - queries;
 - event normalization;
 - solver adapters;
 - snapshots;
 - units;
 - debug data.
-

49. Solver Packages

@four/physics-rapier

@four/physics-box2d

@four/physics-matter

@four/physics-cannon

Each solver package implements the shared adapter interface and declares capability differences.

Part VIII - Implementation Plan

50. Phase 0 - Project Foundation

Deliverables

README.md
LICENSE
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SPECIFICATION.md
IMPLEMENTATION_PLAN.md
ROADMAP.md
package.json
pnpm-workspace.yaml
tsconfig.base.json
eslint.config.js
.github/workflows/ci.yml

Exit criteria

- monorepo installs successfully;
 - all packages compile;
 - tests run;
 - documentation builds;
 - example application starts.
-

51. Phase 1 - Math, Scene, and Time

Components

- Vector2, Vector3, Vector4;
- Matrix3 and Matrix4;
- Quaternion;
- Transform;
- Node;
- Group;
- Scene;
- EventEmitter;
- Clock;
- TimeState;
- fixed-step scheduler;
- dirty transform propagation.

Tests

- hierarchy insertion and removal;
- cycle prevention;
- matrix composition;
- world-transform propagation;
- fixed-step accumulator;
- pause and time scaling;
- interpolation alpha.

Exit criteria

A scene graph can be deterministically stepped without a renderer.

52. Phase 2 - Motion Foundation

Components

- MotionComponent;
- velocity and acceleration integration;
- kinematic controller;
- path following;
- trajectories;
- spring motion;
- transform authority;
- interpolation buffers.

Demonstration

A set of 2D and 3D objects move through:

- constant velocity;
- constant acceleration;
- circular paths;
- spline paths;
- damped spring motion.

Exit criteria

Motion is deterministic, renderer-independent, and unit tested.

53. Phase 3 - Renderer Foundation

Components

- renderer interface;

- WebGL 2 backend;
- camera projections;
- render list;
- GPU buffers;
- shaders;
- textures;
- viewports;
- interpolation-aware rendering.

Exit criteria

Moving 2D and 3D primitives render smoothly despite fixed-step simulation.

54. Phase 4 - Animation Core

Components

- Tween;
- easing;
- Timeline;
- AnimationClip;
- AnimationTrack;
- property binding;
- playback controls;
- event markers;
- deterministic evaluation.

Exit criteria

Any numeric, vector, quaternion, color, or transform property can be animated.

55. Phase 5 - Physics API and First Solver Adapter

Recommended first solver

Use Rapier as the first 2D/3D adapter because it provides modern WebAssembly-based rigid-body physics and parallel conceptual coverage for both dimensions.

Components

- PhysicsWorld;
- RigidBody;
- Collider;
- PhysicsMaterial;

- body types;
- force and impulse APIs;
- collision events;
- ray casting;
- scene synchronization;
- fixed-step integration;
- debug drawing.

Exit criteria

A mixed 2D/3D demonstration supports gravity, collisions, impulses, and sensors through the common API.

56. Phase 6 - Joints and Constraints

Components

- fixed joint;
- distance joint;
- spring;
- hinge;
- slider;
- spherical joint;
- motors;
- limits;
- break thresholds.

Demonstration

An engineering mechanism containing:

- rotating shaft;
- hinge;
- slider;
- spring;
- motor;
- limit switches.

Exit criteria

Constraints remain stable under expected real-time loads.

57. Phase 7 - Physics-Animation Integration

Components

- motion authority;
- animation target poses;
- kinematic-to-dynamic transitions;
- ragdoll activation;
- blended poses;
- root motion;
- physical constraints on animated objects.

Exit criteria

A character or machine can move between animated, kinematic, and physical control without abrupt discontinuities.

58. Phase 8 - Advanced Motion

Components

- steering;
- flocking;
- path planning adapters;
- inverse kinematics;
- trajectory prediction;
- spring-damper controllers;
- PID controller utility;
- robotic joint commands.

Engineering relevance

A PID utility should support simulation and visualization of control systems:

```
const controller = new Four.PIDController({  
  kp: 2,  
  ki: 0.5,  
  kd: 0.1,  
  outputLimits: [-10, 10]  
});
```

59. Phase 9 - Particles and GPU Motion

Components

- particle emitter;

- CPU particle simulation;
- GPU compute simulation;
- force fields;
- collision options;
- lifetime curves;
- trails.

Exit criteria

At least 100,000 simple particles can be simulated and rendered at interactive rates on suitable hardware.

60. Phase 10 - Replay, Snapshots, and Diagnostics

Components

- physics snapshots;
- input recording;
- replay;
- deterministic checksums;
- slow-motion inspection;
- frame stepping;
- collision visualization;
- constraint visualization;
- center-of-mass display;
- velocity and force vectors;
- solver statistics.

Exit criteria

A physics defect can be captured, replayed, and inspected frame by frame.

Part IX - Public API Examples

61. Basic Animated Object

```
import * as Four from "four";

const app = new Four.Application({
  canvas: document.querySelector("canvas"),
  renderer: "auto"
});

const circle = new Four.Circle({
```

```

    radius: 40,
    fill: "#4f7cff"
  });

  app.scene.add(circle);

  Four.animate(circle.position)
    .to({ x: 500 }, 1500)
    .ease("spring")
    .play();

  app.start();

```

62. Dynamic Physics Object

```

const ball = new Four.Mesh({
  geometry: new Four.SphereGeometry({ radius: 0.5 }),
  material: new Four.StandardMaterial({
    baseColor: "#ff8844"
  })
});

ball.addComponent(
  new Four.RigidBody({
    type: "dynamic",
    mass: 1
  })
);

ball.addComponent(
  new Four.SphereCollider({
    radius: 0.5,
    restitution: 0.8,
    friction: 0.3
  })
);

app.scene.add(ball);

```

63. Motorized Hinge

```

const hinge = new Four.HingeJoint({
  bodyA: frameBody,

```

```

    bodyB: rotorBody,
    anchor: new Four.Vector3(0, 0, 0),
    axis: new Four.Vector3(0, 0, 1),
    motor: {
        enabled: true,
        targetVelocity: 10,
        maxTorque: 100
    }
});

physicsWorld.addJoint(hinge);

```

64. Physics and Animation Blend

```

robot.motionAuthority = "blended";

robot.animation.play("walk");
robot.physicsWeight = 0.2;

robot.on("impact", () => {
    robot.physicsWeight = 1;
    robot.animationWeight = 0;
});

```

Part X - Flagship Demonstrations

65. “One Scene, Everything Moves”

The first public demonstration should contain:

- a rotating 3D cube;
- a 2D vector orbit;
- a spring-connected pendulum;
- a bouncing rigid body;
- a world-space label;
- a screen-space control panel;
- a timeline;
- a motorized hinge;
- collision events;
- pause, slow motion, and single-step controls.

Success criterion:

It must feel like one motion-capable engine, not a graphics library with physics bolted on afterward.

66. Engineering Demonstration

Electric Motor Digital Twin

Features:

- 3D motor model;
- animated rotor;
- torque and angular-velocity visualization;
- bearing constraints;
- motorized shaft;
- vibration simulation;
- temperature indicators;
- waveform charts;
- fault injection;
- PID speed controller;
- pause and replay;
- force and torque vector overlays.

This example establishes four.js as useful for engineering, education, simulation, and digital twins.

Part XI - Revised MVP

67. MVP Requirements

The first meaningful release shall include:

Scene

- Node;
- Group;
- Scene;
- Transform;
- Cameras;
- Layers.

Time and Motion

- Clock;
- fixed-step scheduler;

- MotionComponent;
- velocity;
- acceleration;
- path motion;
- interpolation.

Animation

- Tween;
- easing;
- Timeline;
- transform tracks.

Physics

- PhysicsWorld;
- 2D and 3D world descriptors;
- static, dynamic, and kinematic bodies;
- basic colliders;
- gravity;
- forces;
- impulses;
- collision events;
- ray casting;
- one solver adapter;
- debug drawing.

Rendering

- WebGL 2;
- 2D primitives;
- basic 3D meshes;
- lighting;
- sprites;
- text.

Interaction

- pointer events;
- 2D picking;
- 3D ray casting;
- dragging.

Tooling

- tests;
- examples;

- API documentation;
 - benchmark harness;
 - deterministic simulation tests.
-

Part XII - Final Design Statement

four.js should not merely answer:

Where is this object, and how should it look?

It must also answer:

How does this object change through time, what controls its motion, what forces act on it, and how does it interact physically with the rest of the scene?

The defining model is:

```
Object
+-- Transform
+-- Appearance
+-- Motion
+-- Animation
+-- Physics
+-- Interaction
```

The defining promise is:

Create once. Position anywhere. Animate naturally. Simulate physically. Render everywhere.